Operating Systems Lab Manual

Index of Experiments

- Practical 2A
- Practical 2B
- Practical 3A
- Practical 3B
- Practical 4
- Practical 5
- Practical 6
- Practical 7A
- Practical 7B
- Practical 8
- Practical 9
- Practical 10
- Practical 11
- Practical 12

PRACTICAL - 2A

Aim:
To write a shell script to check whether an entered string or a number is a palindrome.

Name:
String and Number Palindrome Checker

Theory:
A palindrome is a sequence of characters or digits that reads the same forward and backward (e.g., "radar" or "12321"). In shell scripting, checking for a palindrome involves reversing the given input and performing a string comparison against the original value.

Algorithm:

- Read the input string or number from the user.
- Reverse the input (using the built-in `rev` command for efficiency).
- Compare the original input with the reversed output.
- If they are equal, print that it is a palindrome; otherwise, print that it is not.

Code:

```
#!/bin/bash

# [Boilerplate] Prompt user for input
read -p "Enter a string or number: " input

# Reverse the input
reversed=$(echo "$input" | rev)

# Core Logic: Compare strings
if [ "$input" == "$reversed" ]; then
    echo "Result: '$input' is a Palindrome."
else
    echo "Result: '$input' is NOT a Palindrome."
fi
```

Output:

```
$ ./palindrome_checker.sh
Enter a string or number: radar
Result: 'radar' is a Palindrome.

$ ./palindrome_checker.sh
Enter a string or number: 12345
Result: '12345' is NOT a Palindrome.
```

Conclusion:
The shell script successfully determines whether an entered string or number is a palindrome by effectively reversing it and performing a string comparison.

PRACTICAL - 2B

Aim:

To write a shell script that generates the first n Fibonacci numbers.

Name:

Fibonacci Sequence Generator

Theory:

The Fibonacci sequence is a series of numbers where the next number is found by adding up the two numbers before it. Generating this computationally requires maintaining two variables that represent the previous two terms, printing the current term, and iteratively updating the variables.

Algorithm:

- Read the desired number of terms (n) from the user.
- Initialize the first two terms: `a = 1` and `b = 1`.
- Start a loop that runs n times.
- In each iteration, print the current value of `a`.
- Calculate the next term (`next = a + b`).
- Update the variables (`a = b` and `b = next`).

Code:

```
#!/bin/bash

read -p "Enter the number of terms (n): " n

# Initialize first two terms
a=1
b=1

echo -n "Fibonacci Series: "

# Core Logic: Loop n times to calculate and print
for (( i=0; i<n; i++ ))
do
    echo -n "$a "
    next=$((a + b))
    a=$b
    b=$next
done
```

Output:

```
$ ./fibonacci_generator.sh
Enter the number of terms (n): 7
Fibonacci Series: 1 1 2 3 5 8 13

$ ./fibonacci_generator.sh
Enter the number of terms (n): 10
Fibonacci Series: 1 1 2 3 5 8 13 21 34 55
```

Conclusion:

The shell script successfully generates the Fibonacci sequence up to the user-specified number of terms using an iterative loop and variable reassignment.

PRACTICAL - 3A

Aim:
To write a shell script to find the smallest and largest among three given numbers.

Name:
Minimum and Maximum of Three Numbers

Theory:
Finding the minimum and maximum involves sequentially comparing variables. In Bash, conditional statements (`if`) combined with arithmetic comparison operators (such as `-lt` for less than and `-gt` for greater than) are used to evaluate the relative sizes of the inputs.

Algorithm:

- Read three numbers (`a`, `b`, and `c`) from the user.
- Assume the first number `a` is both the smallest (`min`) and largest (`max`).
- Compare the second number `b` with `min` and `max`, updating them if necessary.
- Compare the third number `c` with `min` and `max`, updating them if necessary.
- Print the final `min` and `max` values.

Code:

```
#!/bin/bash

# [Boilerplate] Prompt user for input
read -p "Enter three numbers separated by spaces: " a b c

# Core Logic: Initialize min and max to the first number
min=$a
max=$a

# Compare with the second number
if [ $b -lt $min ]; then min=$b; fi
if [ $b -gt $max ]; then max=$b; fi

# Compare with the third number
if [ $c -lt $min ]; then min=$c; fi
if [ $c -gt $max ]; then max=$c; fi

# [Boilerplate] Print the results
echo "Smallest number: $min"
echo "Largest number:  $max"
```

Output:

```
$ ./min_max.sh
Enter three numbers separated by spaces: 10 25 5
Smallest number: 5
Largest number: 25

$ ./min_max.sh
Enter three numbers separated by spaces: −3 0 7
Smallest number: −3
Largest number: 7
```

Conclusion:

The shell script successfully finds the minimum and maximum of three given numbers by utilizing sequential conditional comparisons.

PRACTICAL - 3B

Aim:
To write a shell script that extracts and displays the digits located at odd positions (1st, 3rd, and 5th) in a given 6-digit number.

Name:
Odd Position Digits Extractor

Theory:
String manipulation in Bash allows for extracting specific characters using parameter expansion. The syntax `${string:position:length}` extracts a substring of a specified length starting from a specific zero-based index. Therefore, the 1st, 3rd, and 5th positions correspond to indices 0, 2, and 4.

Algorithm:

- Read a 6-digit number from the user.
- Extract the character at index 0 (1st position).
- Extract the character at index 2 (3rd position).
- Extract the character at index 4 (5th position).
- Print the extracted digits.

Code:

```
#!/bin/bash

# [Boilerplate] Prompt user for input
read -p "Enter a 6-digit number: " num

# Core Logic: Use parameter expansion to extract characters by
index (0-based)
pos1=${num:0:1}
pos3=${num:2:1}
pos5=${num:4:1}

# [Boilerplate] Print the results
echo "Digits at odd positions (1st, 3rd, 5th): $pos1, $pos3,
$pos5"
```

Output:

```
$ ./odd_digits.sh
Enter a 6-digit number: 123456
Digits at odd positions (1st, 3rd, 5th): 1, 3, 5

$ ./odd_digits.sh
Enter a 6-digit number: 987654
Digits at odd positions (1st, 3rd, 5th): 9, 7, 5
```

Conclusion:
The shell script successfully extracts and prints the digits at the 1st, 3rd, and 5th positions from a 6-digit number by utilizing Bash's built-in string parameter expansion capabilities.

PRACTICAL - 4

Aim:
To write a shell script to create a menu-driven calculator using the case statement.

Name:
Menu-Driven Calculator

Theory:
A menu-driven script presents a list of options to the user and executes specific commands based on their selection. The `case` statement in shell scripting is an efficient multi-way branching control structure, making it ideal for mapping user inputs (like choosing an arithmetic operation) to their corresponding executable blocks without using messy, deeply nested if-elif statements.

Algorithm:

- Prompt the user to enter two numbers.
- Display a menu of arithmetic operations (Addition, Subtraction, Multiplication, Division).
- Read the user's choice.
- Use a `case` statement to match the choice and perform the respective arithmetic operation.
- Store and display the result.

Code:

```
#!/bin/bash

# [Boilerplate] Prompt for two input numbers
read -p "Enter first number: " a
read -p "Enter second number: " b

# [Boilerplate] Display options and read choice
echo -e "\nOptions: 1) Add 2) Subtract 3) Multiply 4) Divide"
read -p "Enter your choice (1-4): " ch

# Core Logic: Switch case for arithmetic operations
case $ch in
    1) res=$((a + b)) ;;
    2) res=$((a - b)) ;;
    3) res=$((a * b)) ;;
    4) res=$((a / b)) ;; # Note: Performs integer division
    *) echo "Invalid choice"; exit 1 ;; # [Boilerplate] Error
handling
esac

# [Boilerplate] Print output
echo "Result: $res"
```

OUTPUT:

```
$ ./calculator.sh
Enter first number: 12
Enter second number: 4

Options: 1) Add 2) Subtract 3) Multiply 4) Divide
Enter your choice (1-4): 3
Result: 48

$ ./calculator.sh
Enter first number: 10
Enter second number: 3

Options: 1) Add 2) Subtract 3) Multiply 4) Divide
Enter your choice (1-4): 4
Result: 3
```

Conclusion:

The shell script successfully implements a menu-driven calculator, utilizing the `case` statement to efficiently route user input to the correct arithmetic operation and display the computed result.

PRACTICAL - 5

Aim:
To write a shell script that prints the names of all subdirectories present in the current directory.

Name:
Subdirectory Lister

Theory:
In Linux/UNIX systems, directories are treated as a special type of file. To filter and display only directories, a script can iterate through all items in a given path and use the file test operator `-d` to check if each item is a directory. Alternatively, shell globbing (like `*/`) can be used to match directories directly.

Algorithm:

- Display a header message indicating the current directory.
- Start a loop to iterate through all items (*) in the current directory.
- For each item, use the conditional test `-d` to check if it is a directory.
- If the condition is true, print the name of the directory.

Code:

```bash
#!/bin/bash

# [Boilerplate] Print header
echo "Subdirectories in current directory ($(pwd)):"
echo "--------------------------------------------"

# Core Logic: Loop through all items and check if they are
directories
for item in *; do
    if [ -d "$item" ]; then
        echo "$item"
    fi
done
```

OUTPUT:

```
$ ls -l
drwxr-xr-x 2 user user 4096 Oct 24 10:00 backups
-rw-r–r– 1 user user 150 Oct 24 10:05 data.txt
drwxr-xr-x 4 user user 4096 Oct 24 09:30 projects
-rwxr-xr-x 1 user user 320 Oct 24 10:15 subdir_lister.sh

$ ./subdir_lister.sh
Subdirectories in current directory (/home/user/workspace):
————————————————-
backups
projects
```

Conclusion:

The shell script successfully lists all subdirectories within the current working directory by iterating over its contents and utilizing the `-d` test operator to accurately filter out standard files.

PRACTICAL - 6

Aim:
To write a shell script to count the number of regular files, directories, and symbolic links in a specified directory.

Name:
Directory Contents Counter

Theory:
In Unix-like operating systems, various file types exist. Shell scripts can differentiate between them using conditional file test operators: `-L` checks for a symbolic link, `-f` checks for a regular file, and `-d` checks for a directory. It is important to test for symbolic links first, as a link pointing to a file or directory will also evaluate as true for `-f` or `-d`.

Algorithm:

- Accept a directory path from the user.
- Validate if the given path exists and is a directory.
- Initialize counters for files, directories, and links to zero.
- Iterate through the contents of the directory using a loop.
- Use file test operators to categorize each item and increment the appropriate counter.
- Display the final tallies.

Code:

```
#!/bin/bash

# [Boilerplate] Prompt user and validate input
read -p "Enter directory path: " target_dir

if [ ! -d "$target_dir" ]; then
    echo "Error: '$target_dir' is not a valid directory."
    exit 1
fi

# Initialize counters
f_count=0
d_count=0
l_count=0

# Core Logic: Iterate and classify
for item in "$target_dir"/*; do
    # Skip if directory is empty (prevents '*' from being
treated as a file)
    [ -e "$item" ] || [ -L "$item" ] || continue

    # Note: Check for links FIRST, as links to files also pass
the -f test
    if [ -L "$item" ]; then
        ((l_count++))
    elif [ -f "$item" ]; then
        ((f_count++))
    elif [ -d "$item" ]; then
        ((d_count++))
    fi
done

# [Boilerplate] Print the results
echo "Statistics for $target_dir:"
echo "----------------------------"
echo "Regular Files  : $f_count"
echo "Directories    : $d_count"
echo "Symbolic Links : $l_count"
```

OUTPUT:

```
$ ./dir_counter.sh
Enter directory path: /invalid_path
Error: '/invalid_path' is not a valid directory.

$ ./dir_counter.sh
Enter directory path: /var/log
Statistics for /var/log:
----------
Regular Files : 42
Directories : 12
Symbolic Links : 3
```

Conclusion:

The shell script successfully counts and categorizes the contents of a user-specified directory into regular files, subdirectories, and symbolic links by combining an iterative loop with sequential file test operators.

PRACTICAL - 7A

Aim:
To write a shell script to reverse the contents of a file.

Name:
File Content Reverser

Theory:
In Linux, reversing a file typically means reversing the order of its lines (printing from the last line to the first). The `tac` command (which is `cat` spelled backward) is a built-in utility specifically designed for this purpose. If reversing the actual characters on each line is required instead, the `rev` command is used.

Algorithm:

- Prompt the user to enter the name of the file.
- Verify that the file exists and is a regular file.
- If it exists, use the `tac` command to read and print the file's lines in reverse order.
- If it does not exist, display an error message.

Code:

```bash
#!/bin/bash

# [Boilerplate] Prompt user for filename
read -p "Enter the filename to reverse: " file

# Check if file exists to prevent errors
if [ ! -f "$file" ]; then
    echo "Error: File '$file' does not exist."
    exit 1
fi

# [Boilerplate] Print header
echo "--- Reversed Contents ---"

# Core Logic: Use 'tac' to reverse lines
# (Note: use 'rev "$file"' to reverse characters instead)
tac "$file"
```

OUTPUT:

```
$ cat sample.txt
First Line
Second Line
Third Line

$ ./reverser.sh
Enter the filename to reverse: invalid_file.txt
Error: File 'invalid_file.txt' does not exist.

$ ./reverser.sh
Enter the filename to reverse: sample.txt
— Reversed Contents —
Third Line
Second Line
First Line
```

Conclusion:

The shell script successfully reverses the contents of a specified file by utilizing the `tac` utility to print its lines in reverse order, while also implementing standard file existence validation.

PRACTICAL - 7B

Aim:

To write a menu-driven shell script to copy, remove, and move a file in Linux.

Name:

Menu-Driven File Manager

Theory:

Interactive shell scripts can serve as wrappers for standard Unix commands to make them more user-friendly. This script utilizes an infinite `while` loop combined with a `case` statement to continuously present a menu. Based on the selection, it executes the corresponding file operation commands: `cp` (copy), `rm` (remove), or `mv` (move).

Algorithm:

- Start an infinite loop to display the menu.
- Display options: 1) Copy, 2) Remove, 3) Move, 4) Exit.
- Read the user's choice.
- Using a `case` statement:
  - For option 1, read source and destination paths, then execute `cp`.
  - For option 2, read the target file path, then execute `rm`.
  - For option 3, read source and destination paths, then execute `mv`.
  - For option 4, exit the script.
- Repeat until the user selects the exit option.

Code:

```
#!/bin/bash

# Core Logic: Infinite loop to keep the menu active until exit
while true; do
    # [Boilerplate] Display menu options
    echo -e "\n--- File Operations ---"
    echo "1. Copy a file"
    echo "2. Remove a file"
    echo "3. Move a file"
    echo "4. Exit"
    read -p "Enter your choice (1-4): " choice

    # Switch case to handle operations
    case $choice in
        1)
            read -p "Enter source file: " src
            read -p "Enter destination: " dest
            cp "$src" "$dest" && echo "Success: File copied."
            ;;
        2)
            read -p "Enter file to remove: " file
            rm "$file" && echo "Success: File removed."
            ;;
        3)
            read -p "Enter source file: " src
            read -p "Enter destination: " dest
            mv "$src" "$dest" && echo "Success: File moved."
            ;;
        4)
            exit 0 # Exit the script successfully
            ;;
        *)
            echo "Invalid choice. Please select 1-4." #
[Boilerplate] Error handling
            ;;
    esac
done
```

OUTPUT:

```
$ ./file_manager.sh

— File Operations —
1. Copy a file
2. Remove a file
3. Move a file
4. Exit

Enter your choice (1-4): 1
Enter source file: data.txt
Enter destination: data_backup.txt
Success: File copied.

— File Operations —
1. Copy a file
2. Remove a file
3. Move a file
4. Exit

Enter your choice (1-4): 4
$
```

Conclusion:

The shell script successfully creates a user-friendly, interactive file manager using an infinite loop and a `case` statement to execute standard Unix file operations.

PRACTICAL - 8

Aim:

To write a shell script to monitor disk usage and generate an alert if any partition exceeds 80% usage.

Name:

Disk Space Monitoring and Alert Script

Theory:

System administrators regularly monitor disk space to prevent system crashes. The `df` (disk free) command reports file system disk space usage. By piping the output of `df` into text processing tools like `awk`, a script can isolate the percentage usage of each partition, perform numerical comparisons against a threshold (like 80%), and trigger automated alerts.

Algorithm:

- Define the maximum allowable disk usage threshold (80).
- Execute the `df -h` command to retrieve disk usage statistics in a human-readable format.
- Pipe the output to `awk` to process it line by line.
- Skip the first line (the header).
- Extract the usage percentage (5th column) and convert it to a numeric value.
- If the usage is greater than or equal to the threshold, print an alert specifying the partition name (1st column) and its current usage.

Code:

```
#!/bin/bash

THRESHOLD=80

# [Boilerplate] Print startup message
echo "Monitoring disk usage (Alert threshold: >= $THRESHOLD%)..."
echo "-----------------------------------------------------------"

# Core Logic: One-liner using df and awk for maximum conciseness
# NR>1: Skips the header row
# $5+0: Implicitly converts the percentage string (e.g., "82%")
# to a number (82)
# -v t=$THRESHOLD: Passes the bash variable into awk

df -h | awk -v t="$THRESHOLD" 'NR>1 && $5+0 >= t {
    print "CRITICAL ALERT: Partition "$1" is at "$5" usage!"
}'
```

OUTPUT:

```
$ ./disk_monitor.sh
Monitoring disk usage (Alert threshold: >= 80%)...
————————————————————
CRITICAL ALERT: Partition /dev/sda1 is at 85% usage!
CRITICAL ALERT: Partition /dev/sdb2 is at 81% usage!

$ (Example when all partitions are under threshold)
$ ./disk_monitor.sh
Monitoring disk usage (Alert threshold: >= 80%)...
————————————————————
$
```

Conclusion:

The shell script successfully monitors system disk usage by processing `df` command output through `awk`, identifying and alerting the user only when partitions reach or exceed the defined critical threshold.

PRACTICAL - 9

Aim:

To write a shell script that translates capital letters to small letters and small letters to capital letters using the awk command.

Name:

Case Inversion using AWK

Theory:

While the `tr` command is normally the go-to tool for character swapping in shell scripts, the `awk` text-processing language can also achieve this programmatically. It does so by isolating each character, matching it against regular expressions (`/[A-Z]/` or `/[a-z]/`), and dynamically applying the built-in string functions `tolower()` and `toupper()`.

Algorithm:

- Read an input string from the user.
- Pipe the string into `awk`.
- Within `awk`, initiate a loop that runs for the length of the string.
- Extract each character one by one using the `substr()` function.
- Check the character's case. If uppercase, convert to lowercase. If lowercase, convert to uppercase.
- Concatenate the converted characters and print the final result.

Code:

```
#!/bin/bash

# [Boilerplate] Prompt user for input
read -p "Enter a string to toggle case: " input
echo -n "Converted string: "

# Core Logic: awk script to iterate through characters and swap
cases
echo "$input" | awk '{
    out = ""
    for (i=1; i<=length($0); i++) {
        c = substr($0, i, 1)
        if (c ~ /[A-Z]/) {
            out = out tolower(c)
        } else if (c ~ /[a-z]/) {
            out = out toupper(c)
        } else {
            out = out c # Keep numbers and special characters
as is
        }
    }
    print out
}'
```

OUTPUT:

```
$ ./case_toggle.sh
Enter a string to toggle case: Hello World!
Converted string: hELLO wORLD!

$ ./case_toggle.sh
Enter a string to toggle case: BASH script 123
Converted string: bash SCRIPT 123
```

Conclusion:

The shell script successfully performs case inversion on a given string by using `awk` to iterate through individual characters and apply conditional transformations based on regular expression matching.

PRACTICAL - 10

Aim:

To write a C program to create a new process using the fork() system call and compile it using the GCC compiler.

Name:

Process Creation using fork()

Theory:

In UNIX/Linux operating systems, a new process is created using the `fork()` system call. The newly created process is called the child process, which is an almost exact duplicate of the calling process (the parent). `fork()` returns a process ID (PID): it returns 0 to the child process, a positive integer (the child's PID) to the parent, and a negative value if process creation fails.

Algorithm:

- Include the necessary standard libraries (`<stdio.h>`, `<unistd.h>`, `<sys/types.h>`).
- Declare a variable of type `pid_t` to store the process ID.
- Call the `fork()` function and assign its return value to the variable.
- Use an `if-else` block to evaluate the return value:
  - If the value is less than 0, print an error message (Fork failed).
  - If the value equals 0, print a message identifying as the child process, displaying its PID (`getpid()`) and its parent's PID (`getppid()`).
  - If the value is greater than 0, print a message identifying as the parent process, displaying its own PID and the newly created child's PID.
- Exit the program.

Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    // [Boilerplate] Call fork() to create a child process
    pid = fork();

    // Core Logic: Determine execution context based on fork's
return value
    if (pid < 0) {
        // Fork failed
        fprintf(stderr, "Error: Process creation failed!\n");
        return 1;
    }
    else if (pid == 0) {
        // Child process context
        printf("CHILD PROCESS  -> My PID: %d | Parent's PID:
%d\n",
               getpid(), getppid());
    }
    else {
        // Parent process context
        printf("PARENT PROCESS -> My PID: %d | Created Child's
PID: %d\n",
               getpid(), pid);
    }

    return 0;
}
```

OUTPUT:

```
$ gcc process.c -o process
$ ./process
PARENT PROCESS -> My PID: 1245 | Created Child's PID: 1246
CHILD PROCESS -> My PID: 1246 | Parent's PID: 1245
```

Conclusion:

The C program successfully demonstrates process creation using the `fork()` system call. By evaluating the return value of `fork()`, the program was able to distinguish between the parent and child execution contexts and print their respective Process IDs.

PRACTICAL - 11

Aim:
To study and practice standard Linux commands used for file and directory management, file manipulation, and process management.

Name:
Study of Basic Linux Commands

Theory:
An Operating System provides a command-line interface (CLI) to interact with the system kernel. In Linux, the shell interprets these commands to perform system-level tasks. Understanding these commands is fundamental for navigating the file system, manipulating data, and controlling running processes.

COMMAND SUMMARY:

Command	Description	Example Usage
`pwd`	Displays the absolute path of the current directory.	`pwd`
`ls`	Lists contents of a directory. Options: `-l` (long), `-a` (all).	`ls -la`
`cd`	Changes the current working directory.	`cd ..` or `cd ~`
`mkdir`	Creates a new, empty directory. `-p` creates parents.	`mkdir -p new/dir`
`rmdir`	Deletes a directory (must be empty).	`rmdir old_docs`
`touch`	Updates timestamps or creates a new empty file.	`touch notes.txt`
`mv`	Moves or renames files and directories.	`mv old.txt new.txt`

Command	Description	Example Usage
`cat`	Displays file content or concatenates files.	`cat file1 file2`
`cut`	Extracts specific sections/columns from lines.	`cut -d',' -f1 data.csv`
`sort`	Sorts lines alphabetically or numerically (`-n`).	`sort -r names.txt`
`wc`	Counts lines (`-l`), words (`-w`), and bytes.	`wc -l report.txt`

Command	Description	Example Usage
`ps`	Displays snapshot of current processes.	`ps aux`
`kill`	Sends a signal (like SIGKILL `-9`) to a PID.	`kill -9 1234`

OUTPUT (Sample Execution):

```
$ pwd
/home/user/lab

$ mkdir test_dir && cd test_dir
$ touch file1.txt file2.txt
$ ls
file1.txt file2.txt
```

Conclusion:

Through this study, the fundamental Linux commands for system interaction were explored. Proficiency in these commands allows for efficient management of the file system and system resources directly through the shell.

PRACTICAL - 12

Aim:
To study and practice Linux commands used for user and permissions management, networking, system monitoring, and general utilities.

Name:
Study of System, User, and Network Commands

Theory:
System administration and daily operations in Linux require a deep understanding of user privileges, network connectivity, and system resources. This suite of commands provides the necessary tools to secure files, diagnose network issues, monitor system health, and utilize general shell features efficiently.

COMMAND SUMMARY:

Command	Description	Example Usage
`who`	Lists users currently logged into the system.	`who`
`whoami`	Prints the effective username of the current user.	`whoami`
`chmod`	Changes access permissions (Read/Write/Execute).	`chmod 755 script.sh`
`chown`	Changes file/directory owner and group.	`sudo chown user:grp file`

Command	Description	Example Usage
`ping`	Tests reachability of a host via ICMP packets.	`ping -c 4 google.com`
`ifconfig`	Legacy tool to view/configure network interfaces.	`ifconfig`
`ip a`	Modern tool to show network device addresses.	`ip a`

Command	Description	Example Usage
`uptime`	Shows how long system is up and load average.	`uptime`
`free`	Displays amount of free and used memory (`-h`).	`free -h`
`uname`	Prints system information and kernel version.	`uname -a`

Command	Description	Example Usage

`echo`	Prints text or variable values to the screen.	`echo $USER`
`date`	Displays or sets the system date and time.	`date "+%D"`
`history`	Shows the list of previously executed commands.	`history`
`man`	Opens the manual/reference for a command.	`man chmod`
`alias`	Creates a shortcut for a longer command.	`alias ll="ls -la"`

OUTPUT (Sample Execution):

```
$ whoami
student

$ uptime

$ free -h
total used free shared buff/cache available
Mem: 7.7Gi 1.2Gi 4.5Gi 156Mi 2.0Gi 6.2Gi

$ ping -c 1 127.0.0.1
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.032 ms

— 127.0.0.1 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
```

Conclusion:

This experiment provided hands-on experience with administrative and diagnostic Linux commands. These tools are essential for managing user environments, monitoring system resources, and troubleshooting network connectivity.